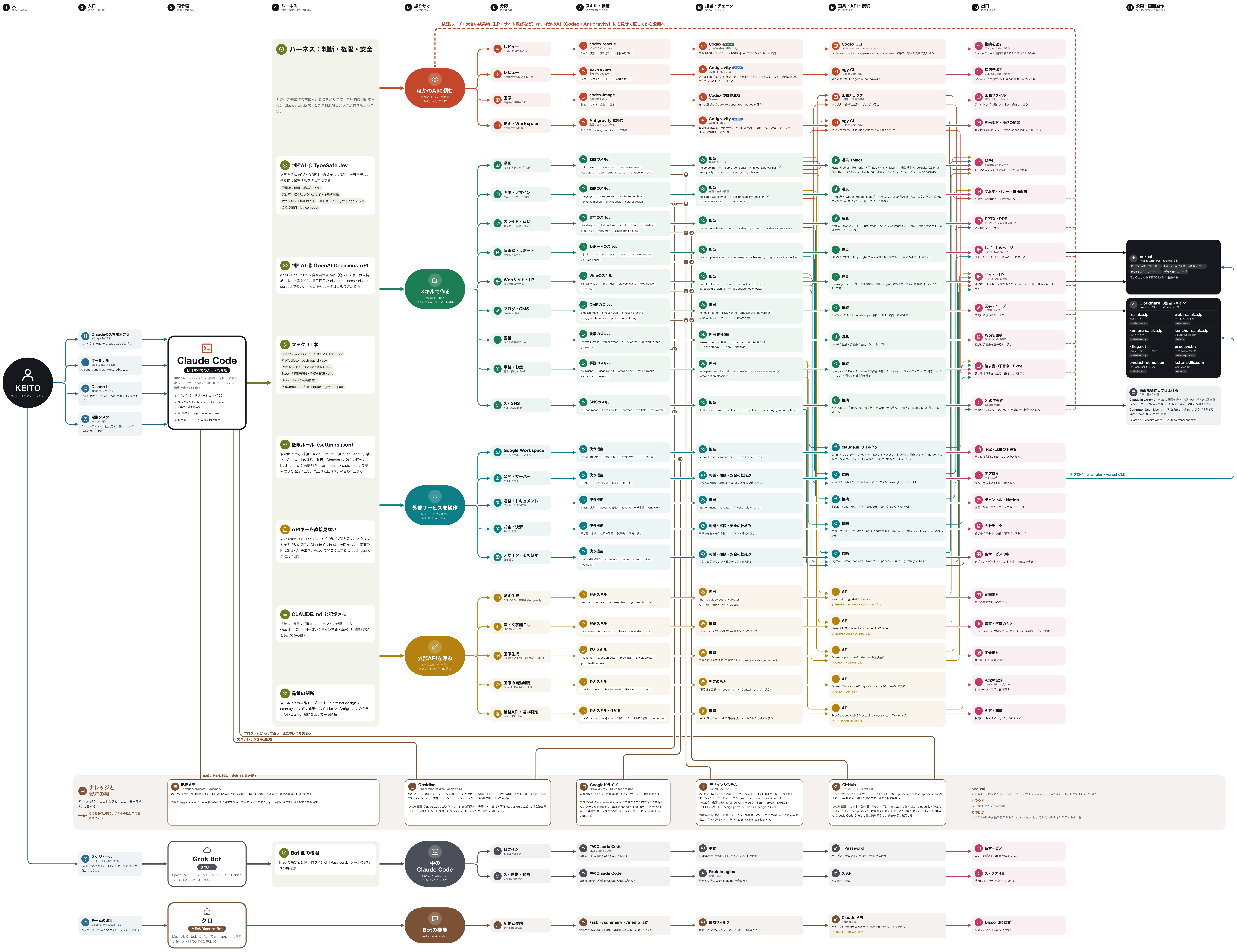
KEITOのAIエコシステム

左から右へ11列でたどれます。ほとんどの依頼は Claude Code が受け、ハーネス（判断AI・フック・権限・APIキーの隔離）を通したうえで「ほかのAIに頼む」「スキルで作る」「外部サービスを操作する」「外部APIを呼ぶ」に振り分けます。できたものは Cloudflare の独自ドメインと Vercel に公開します。Grok Bot（クラウドPC）とクロ（Mac上で動く別のプログラム）は、Claude Code を通らない別の入口です。下の「ナレッジと資産の棚」（記憶メモ・Obsidian・Googleドライブ・デザインシステム・GitHub）は、Claude Code と各スキルの行から茶色の線でつながっています。2026年11月5日時点の設定ファイルとフォルダから作成しました。

- **Claude Codeがほぼ入口**：スマホのClaudeアプリ・ターミナル・Discordから
- **画像は Codex、動画は Antigravity が基本**：レビューも Codex と Antigravity。動画カットの見直しと Workspace の操作は Antigravity へ
- **APIキーは.envの1か所**：スクリプトが読み、Claude Code は開かない決まり
- **判断AIが2つ**：TypeSafe Jev と OpenAI Decisions API
- **Grok Bot**はスケジュールで動く別の入口、中でClaude Codeを使う
- **公開**は Cloudflare の独自ドメイン8つと Vercel。APIが無い所は画面を操作
- **検証ループ**：大きい成果物（LP・サイト改修など）は「ほかのAIに頼む」へ戻って見直す
- **外部APIが要るスキル**は薄茶の線で「外部APIを呼ぶ」を通って戻る。画像と動画は赤茶の線で「ほかのAIに頼む」へ
- **外部サービスを使うスキル**は青緑の線で「外部サービスを操作」を通って戻る（請求→マネーフォワードなど）
- **ナレッジと資産**は茶色の線。点のある行が、その置き場を読み書きする



1 **人** 頼む・確かめる・決める
2 **入口** どこから頼むか
3 **司令塔** 依頼を受けるAI
4 **ハーネス** 判断・権限・安全の仕組み
5 **振り分け** 4つの行き先
6 **分野** 何を作るか
7 **スキル・機能** どの手順書を使うか
8 **担当・チェック** サブエージェント
9 **道具・API・接続** 何で動かすか
10 **出口** 何ができるか
11 **公開・画面操作** APIで届かない所は画面で

検証ループ：大きい成果物（LP・サイト改修など）は、ほかのAI（Codex・Antigravity）にも見せて直してから公開へ

KEITO
頼む・確かめる・決める

Claudeのスマホアプリ
外出先からの入口
スマホから Mac の Claude Code に頼む

ターミナル
Mac の前にいるとき
Claude Code CLI。作業の大半はここ

Discord
discord プラグイン
発言を受けて Claude Code が返信（プラグイン）

定期タスク
決まった時刻に
AIニュース・メール重要度・半導体ニュース（毎朝7:30）ほか

Claude Code
ほぼすべての入口・司令塔
頭は Claude Opus 5.5（推論 xhigh）。依頼を読み、行き先を決めて仕事を配り、戻ってきた結果をまとめて直す。
- スキル147・サブエージェント150
- プラグイン17（codex・cloudflare・playwright ほか）
- 自作MOD：agents-pane・ja-ui
- 利用額をステータス行に円で表示

ハーネス：判断・権限・安全

どの行き先に進む前にも、ここを通ります。最終的に判断するのは Claude Code で、2つの判断AIとフックが材料を出します。

判断AI ① TypeSafe Jev

文章を読んで0.2〜0.35秒で点数をつける速い分類モデル。送る前に秘密情報を伏せ字にする

- 依頼時：種類・曖昧さ・分岐
- 実行前：取り返しのつかなさ・依頼の範囲
- 締める前：未検証の完了
- 案を選ぶとき：jev-judge で採点
- 会話の圧縮：jev-compact

判断AI ② OpenAI Decisions API

gpt-6-luna で画像を自動判定する網（崩れた文字・個人情報・余白・重なり）。電子冊子の ebook-harness・ebook-spread で使い、引っかかったものは目視で確かめる

フック 11本

- UserPromptSubmit：文体を読む指示・Jev
- PreToolUse：bash-guard・Jev
- PostToolUse：Obsidian登録を促す
- Stop：利用額通知・登録の確認・Jev
- SessionEnd：利用額通知
- PreCompact・SessionStart：jev-compact

権限ルール（settings.json）

既定は auto。**確認**：sudo・rm -rf・git push --force／**禁止**：Chatworkの削除／**許可**：Chatworkのほかの操作。bash-guard が再帰削除・force push・sudo・.env の読み取りを確認に回す。禁止は迂回せず、報告して止まる

APIキーを直接見ない

`~/.claude/skills/.env` の1か所に27個を置く。スクリプトが実行時に読み、Claude Code は中を開かない・画面や図に出さない決まり。Read で開こうとすると bash-guard が確認に回す

CLAUDE.md と記憶メモ

常時ルール5つ（担当エージェントの起動・JLS+・Obsidian CLI・AIっぽいデザイン禁止・Jev）と記憶273件を読んでから動く

品質の関所

スキルごとの検品エージェント → natural-design の scan.py → 大きい成果物は Codex と Antigravity の多モデルレビュー。指摘を直してから納品

ほかのAIに頼む
画像は Codex、動画は Antigravity が基本

- **レビュー** Codexに見てもらう → **codex:rescue** プラグイン（codex） 2本目の実装／原因調査／成果物の見直し → **Codex** OpenAI gpt-6-astra・推論 xhigh スキル138・エージェント106を持つ別のエージェントとして読む → **Codex CLI** codex:rescue・codex exec codex-companion → app-server か、codex exec で呼び、結果の文章を受け取る → **指摘を返す** Claude Code が統合 Claude Code が指摘を取り込んで直してから納品
- **レビュー** Antigravityに見てもらう → **agy-review** 多モデルレビュー 文章／デザイン／コード／動画のカット → **Antigravity** Google Gemini・agy v1.5.1 スキル149（複製）を持つ。照らす原本を指定して見直してもらう。動画に強いので、カットのレビューはここ → **agy CLI** ~/.local/bin/agy スキル置き場は ~/.gemini/config/skills → **指摘を返す** Claude Code が統合 Codex と Antigravity の両方の指摘をまとめて直す
- **画像** 画像生成は基本ここ → **codex-image** 画像生成スキル 挿絵／サムネの素材／図版 → **Codex の画像生成** OpenAI 描いた画像は Codex の generated_images に保存 → **画像チェック** 文字化けを目で確認 文字入りは必ず生成後に1文字ずつ照合 → **画像ファイル** 資料・LP・サムネへ デスクトップの案件フォルダに保存して使う
- **動画・Workspace** Antigravityに頼む → **Antigravity に頼む** 動画は基本ここで作る 動画生成／Google Workspace の操作 → **Antigravity** Google Gemini・agy 動画生成は基本 Antigravity。たまに外部APIで直接作る。Gmail・カレンダー・Drive の操作もここへ頼む → **agy CLI** ~/.local/bin/agy 結果を受け取り、Claude Code が次の工程へつなぐ → **動画素材・操作の結果** 動画は編集に差し込み、Workspace は結果を報告する

スキルで作る
手順書147個と担当のサブエージェント150体

- **動画** カット・テロップ・図解 → **動画のスキル** cut／telop／motion-vault／video-asset-vault／beat-motion-video／zukai-kaisetsu／youtube-long-edit → **担当** 順番にチェック telop-splitter → telop-proofreader → telop-sync-verifier ／ mv-quality-checker ＋ mv-originality-checker → **道具（Mac）** HyperFrames・Remotion・ffmpeg・mlx-whisper。映像は基本 Antigravity（たまに外部API）、声は外部API、曲は Suno（外部サービス）。カットのレビューは Antigravity → **MP4** YouTube・ショート 1秒ごとのコマを目で検品してから書き出し
- **画像・デザイン** サムネ・バナー・漫画 → **画像のスキル** image-gen／x-design-buzz／youtube-thumbnail／yonkoma-manga／thumb-vault／natural-design → **担当** 企画→生成→検査 design-buzz-planner → design-usability-checker ／ yonkoma-planner → yonkoma-qa → **道具** 生成は基本 Codex（codex-image）、一部のスキルは外部APIを呼ぶ。文字入りは生成後に目で照合し、崩れたら作り直すか PIL で重ねる → **サムネ・バナー・投稿画像** X投稿・YouTube・Substack へ
- **スライド・資料** セミナー・研修・提案 → **資料のスキル** realaize-pptx／keito-slides／aoshiro-slides／zukai-slides／slide-vault／mitsumori／simple-notion-slide → **担当** slide-content-researcher → slide-copy-writer → slide-design-reviewer → **道具** pptxの生成スクリプト・LibreOffice・ヘッドレスChromeでPDF化。Notion のスライドは外部サービスを呼ぶ → **PPTX・PDF** デスクトップの案件フォルダ 話す用のノート付き
- **議事録・レポート** 文字起こしから → **レポートのスキル** gijiroku／transcript-report／manekuru-training-report／youtube-kousei → **担当** transcript-analyzer → minutes-quality-checker ／ report-quality-checker → **道具** HTMLを生成し、Playwright で表示崩れを撮って確認。公開は外部サービスを呼ぶ → **レポートのページ** Excel・Notion にも 決まったことだけを「やること」に載せる
- **Webサイト・LP** 番号で組み立てる → **Webのスキル** STYLE-VAULT／lp-builder／oid-seminar-lp／web-builder／keito-lab → **担当** sv-site-planner → 実装 → sv-quality-checker ／ lp-structure-planner → lp-consistency-checker → **道具** Playwright でスマホ・PCを撮影。公開と Figma は外部サービス。画像は Codex か外部APIで作る → **サイト・LP** KEITO LAB に登録 スマホとPCで撮って確かめてから公開。ソースは GitHub 非公開の v-site
- **ブログ・CMS** EmDashのサイト → **CMSのスキル** emdash-ktlog／emdash-web／emdash-procevo／ktlog-youtube-article／procevo-report-blog → **担当** emdash-content-reviewer ＋ emdash-change-verifier 元資料と照合し、プレビューも開いて確認 → **接続** EmDash の MCP・emdash.py。図は HTML で描いて WebP に → **記事・ページ** 下書きが既定 公開は指示があるときだけ
- **書籍** 本ごとの専属チーム → **執筆のスキル** ichiyasa-writer／yabai-writer／ai100-writer／gptwork-writer／gws-writer → **担当 約45体** researcher → 執筆 → tone・format・list を並列 → consistency → docx・obsidian → **道具** Wordの生成・仮画像の生成・Obsidian CLI → **Word原稿** Obsidianの進捗表 図版は仮画像を埋め込んで渡す
- **事務・お金** 請求・売上・メール → **事務のスキル** seikyusho／uriage-report／gmail-digest／mail-furiwake／kanren-kabu-research → **担当** uriage-data-auditor ＋ insight-writer → report-reviewer ／ email-action-classifier → **接続** openpyxl で Excel に。Gmail の操作は基本 Antigravity、マネーフォワードは外部サービス、Jev の判定は外部APIを呼ぶ → **請求書の下書き・Excel** 請求書は下書きで止め、送るのは KEITO
- **X・SNS** KEITOの口調で → **SNSのスキル** ai-news-radar／keito-x-writer／hermes／x-article／newsletter → **担当** keito-news-curator → keito-voice-checker → post-engagement-optimizer → **接続** X News API（xurl）、Hermes 経由で Grok の X検索。下書きは Typefully（外部サービス）へ → **X の下書き** @keitowebai 記事の本文は API で入れ、画像だけ画面操作で入れる

外部サービスを操作
MCP・コネクタ経由。判断は Claude Code

- **Google Workspace** メール・予定・ファイル → **使う機能** メールの仕分け／予定の登録／Driveの検索／シートの更新 → **担当** email-thread-summarizer → email-action-classifier → **claude.ai のコネクタ** Gmail・カレンダー・Drive・ドキュメント・スプレッドシート。操作は基本 Antigravity に頼み（A の行）、ここを通るのはメールの仕分けなど一部のスキル → **予定・返信の下書き** 予定には各回のZoomリンクを入れる
- **公開・サーバー** サイトを出す → **使う機能** デプロイ／ログの確認／DNS／D1・R2 → **判断・権限・安全の仕組み** 本番への反映は依頼の範囲か Jev と権限で確かめてから → **接続** Vercel のコネクタ・Cloudflare のプラグイン・wrangler・vercel CLI → **デプロイ** 本番に反映 反映したら本番を開いて確かめる
- **連絡・ドキュメント** チームとのやり取り → **使う機能** Slackへ投稿／Discordの管理／Notionのページ作成／Chatwork → **担当** notion-manual-validator ／ ccbu-rule-checker → **接続** Slack・Notion のコネクタ、discord-mcp、Chatwork の MCP → **チャンネル・Notion** 講座カリキュラム・マニュアル・ニュース
- **お金・決済** 会計と決済 → **使う機能** 請求書の作成／仕訳の確認／試算表／決済の設定 → **判断・権限・安全の仕組み** 削除や送金にあたる操作はしない・確認に回す → **接続** マネーフォワードの MCP（会計）と請求書API（鍵は .env）・Stripe と 1Password のプラグイン → **会計データ** 請求書の下書き・決算の不明点リストなど
- **デザイン・そのほか** 読み書き → **使う機能** Figmaの読み書き／Supabase／Luma／Zapier／Suno／Typefully → **判断・権限・安全の仕組み** つなぐ先が正しいかを確かめてから書き込む → **接続** Figma・Luma・Zapier のコネクタ、Supabase・Suno・Typefully の MCP → **各サービスの中** デザイン・データ・イベント・曲・投稿の下書き

デプロイ（wrangler・vercel CLI）

外部APIを呼ぶ
キーは .env の1か所。スクリプトが実行時に読む

- **動画生成** たまに直接（基本は Antigravity） → **呼ぶスキル** beat-motion-video／hermes-video／higgsfield 系／fal → **担当** hermes-video-output-validator 尺・比率・壊れたファイルを確認 → **API** Veo・fal・Higgsfield・Runway GEMINI_VEO・FAL・RUNWAYML など → **動画素材** 編集の中の差し込みに使う
- **声・文字起こし** 耳に関わるもの → **呼ぶスキル** motion-vault のナレーション／beat-motion-video／cut → **確認** ElevenLabs の読み間違いは書き起こして確かめる → **API** Gemini TTS・ElevenLabs・OpenAI Whisper ELEVENLABS・OPENAI など → **音声・字幕のもと** ナレーションと文字起こし。曲は Suno（外部サービス）で作る
- **画像生成** 一部のスキルだけ（基本は Codex） → **呼ぶスキル** image-gen／x-design-buzz／lp-builder／STYLE-VAULT／youtube-thumbnail → **確認** 文字入りは生成後に1文字ずつ照合（design-usability-checker） → **API** OpenAI gpt-image-2・Gemini の画像生成 OPENAI・GEMINI など → **画像素材** サムネ・LP・図版に使う
- **画像の自動判定** OpenAI Decisions API → **呼ぶスキル** ebook-harness／ebook-spread／decisions_check.py → **判定のあと** 要確認を目視 → codex_verify（Codexが1文字ずつ照合） → **API** OpenAI Decisions API・gpt-6-luna（画像はbase64で送る） OPENAI_API_KEY → **判定の記録** qa/decisions_*.json 引っかかった図だけ作り直す
- **業務API・速い判定** Jev・LINE ほか → **呼ぶスキル・仕組み** mail-furiwake／jev-judge／判断フック／LINEの配信／Vectorizer → **確認** Jev はフックが3か所で自動採点。メールの振り分けにも使う → **API** TypeSafe Jev・LINE Messaging・Vectorizer・Workers AI TYPESAFE・LINE など → **判定・配信** 報告に「Jev: A 0.80」のように添える

公開・画面操作

Vercel
*.vercel.app ほか、公開先が多数
KEITO LAB（作品一覧）／keitoai.dev（著書・独自ドメイン）／report-○○（レポート）／デモ・資料のページ
新しく出したら KEITO LAB に登録する

Cloudflare の独自ドメイン
EmDash 7サイト＋Workers 1つ

realaize.jp 会社サイト `realaize-cms`	**web.realaize.jp** ホームページ制作 `emdash-web`
komon.realaize.jp AIアドバイザー `emdash-komon`	**kenshu.realaize.jp** Claude Code 研修 `EmDash`
ktlog.net ブログ・ポートフォリオ `emdash-ktlog`	**procevo.biz** Procevo のセミナー `emdash-procevo`
emdash-demo.com EmDash のサンプル集 `emdash-demo`	**keito-skills.com** スキル配布所 `Workers`

画面を操作して仕上げる
APIで届かない所
Claude in Chrome：Web の画面を操作。X記事のエディタに画像を入れる・YouTube の文字起こしを取る・ログインが要る画面を撮る
Computer Use：Mac のアプリを操作して撮る。ブラウザは見るだけなので Web は Chrome 側で
x-article／aoshiro-slides／youtube-transcript-excel

プログラムは git で直し、過去の版にも戻せる
文体ナレッジを毎回読む
依頼のたびに読み、決まりを書き足す

ナレッジと資産の棚

多くの依頼が、ここから読み、ここへ書き戻す5つの置き場

点のある行が使う。点の中の絵は下の置き場と同じ

記憶メモ
~/.claude/projects/⋯/memory
273件。1件に1つの事実を書き、MEMORY.md が目次になる。KEITO の指示と決まり、案件の経緯、接続先のメモ
つながる所 Claude Code が依頼のたびに目次を読み、関係するメモを開く。新しい指示や決まりは1件ずつ書き足す

Obsidian
~/Desktop/Obsidian・obsidian CLI
925ノート。書籍のナレッジ（AI活用100・いちやさ・GWS本・ChatGPT Work本）、スキル一覧（Claude Code 208・Codex 73）、文体ナレッジ、デザインナレッジ、X投稿ネタ帳、メルマガ保管庫
つながる所 Claude Code が文体ナレッジを毎回読み、書籍・X・SNS・画像（x-design-buzz）の行も読み書きする。スキルを作ったり直したりしたときは、フックが一覧への登録を促す

Googleドライブ
Drive コネクタ・Drive for desktop
講座の配布フォルダ、執筆資料のリンク、サテライト動画の企画書
つながる所 Google Workspace のコネクタで配布フォルダを探し、リンクの実在を確かめる（claudecode-curriculum）。指示があれば、企画書をドライブの所定のフォルダへコピーする（satellite-youtube）

デザインシステム
見た目の決まりと素材庫
AoBase（realaize.jp の青）、STYLE VAULT（DS 1,341件・レイアウト200・モーション181）、スライドの型（keito・aoshiro・monoblue・SLIDE VAULT）、動画の素材庫（MOTION・VIDEO ASSET・SHORT EFFECT・THUMB VAULT）、design-parts 77、natural-design 73条項
つながる所 動画・画像・スライド・議事録・Web・ブログの行が、型を番号で選んで色と部品を使い、仕上げに条項と照らして検査する

GitHub
リポジトリ17（非公開14）
v-site（Vercel に出したサイト156フォルダの正本）、procevo-emdash（procevo.biz の正本）、ai-lift ほか。履歴が残るので、過去の版に戻せる
つながる所 スライド・議事録・Web の行は、出したものを v-site に push して控えにする。ブログの行（procevo）は作業前に最新を取り込んでから直す。プログラムの修正は Claude Code が git で直接読み書きし、過去の版にも戻せる

Mac の中
記憶メモ・Obsidian（デスクトップ）・デザインシステム（各スキルと STYLE-VAULT のフォルダ）
クラウド
Googleドライブ・GitHub
このほか
KEITO LAB の台帳や各スキルの registry.json は、それぞれのスキルのフォルダに置く

スケジュール
Grok Bot の自動化機能
時刻を決めておくと、Mac を通らずに Bot が自分で動き出す

Grok Bot
別の入口
SpaceXAI のエージェント。クラウドPC（Debian 13・8コア・15GB）で動く

Bot 側の権限
Mac の設定とは別。ログインは 1Password、ツールの実行は都度確認

中の Claude Code
Bot のPCに導入し、Maxでログイン済み

- **ログイン** 1Passwordで → **中のClaude Code** Maxでログイン済み Bot の中で Claude Code CLI を動かす → **承認** 1Password の承認画面で使うアカウントを確認 → **1Password** サービスへのログインを Bot が代わりに行う → **各サービス** ログインが必要な作業を続けられる
- **X・画像・動画** Grokの得意分野 → **中のClaude Code** 決まった時刻の作業を Claude Code が進める → **Grok Imagine** 画像・動画 画像と動画は Grok Imagine で作られる → **X API** Xの検索・投稿 → **X・ファイル** 結果は Bot のクラウドPCに残る

チームの発言
Discord チームRealAIze
メンバーが @クロ やスラッシュコマンドで頼む

クロ
自作のDiscord Bot
Mac で動く Node のプログラム。launchd で常駐する作り（11/05時点は停止中）

Botの機能
~/discord-kuro-bot

- **記録と要約** チームRealAIze → **/ask・/summary・/memo ほか** 全発言を SQLite に記録し、3時間ごとに取りこぼしを回収 → **権限フィルタ** 質問した人が見られるチャンネルの内容だけ使う → **Claude API** Sonnet 5.5 /ask・/summary のときだけ Anthropic の API を直接呼ぶ ANTHROPIC_API_KEY → **Discordに返信** 根拠リンクと優先度つきの要約